

Daedalus-AI — Comprehensive Project Report

AI Co-Founder Team — A Multi-Agent Startup Validation Platform

1. Project Overview

Daedalus-AI is a multi-agent AI system that simulates a virtual founding team of **6 specialized AI agents** to validate, critique, and plan any startup idea in real-time. Given a raw startup idea, the system orchestrates a team of AI co-founders — Product Manager, UI Designer, Backend Architect, Marketing Strategist, Investor, and Skeptic — who collaboratively generate a complete startup plan through debate and revision.

Core Value Proposition

"Cursor builds your app in minutes. We tell you whether you should build it at all — and how, using a debating AI founding team."

Key Deliverables

Output	Format	Description
Startup Plan	JSON/UI	Complete analysis from all 6 agents
Startup Score	Radar Chart	Feasibility, Market Size, Differentiation, Team Fit
Pitch Deck	.pptx	12-slide investor-ready pitch deck
Wireframe	.html	Interactive SaaS wireframe with Tailwind CSS
Code Skeleton	.zip	FastAPI project scaffold with models, routes, Docker

2. System Architecture

2.1 High-Level Architecture Diagram

```
graph TB
  subgraph "Frontend — React + Vite"
    A["User Input\n(Startup Idea)"] --> B["InputBar Component"]
    B --> C["useSSE Hook"]
    C --> D["POST /build-startup"]
    D --> E["SSE Stream Consumer"]
    E --> F["AgentGrid"]
    F --> G["CritiqueThread"]
    G --> H["ScoreRadar"]
    H --> I["WireframePreview"]
  end
```

```
D --> I["ExportButtons"]
end

subgraph "Backend – FastAPI"
J["FastAPI Server\n(main.py)"] -->|"SSE Stream"| C
J --> K["Orchestrator Pipeline"]
J --> L["Artifact Generators"]
end

subgraph "Orchestrator – LangGraph Pipeline"
K --> M["Domain Classifier"]
M --> N["Round 1: 4 Agents\n(PM → UI, Backend, Marketing)"]
N --> O["Round 2: Critique\n(Investor + Skeptic)"]
O --> P["Revision Loop"]
P --> Q["Scoring Engine"]
end

subgraph "Artifact Generation"
L --> R["Pitch Deck\n(python-pptx)"]
L --> S["Wireframe\n(Claude API / Template)"]
L --> T["Code Skeleton\n(ZIP Generator)"]
end

subgraph "AI Models"
U["Gemini 2.5 Flash\n(Google GenAI)"] --> N
U --> O
U --> P
V["Claude Sonnet\n(Anthropic)"] --> S
end
```

2.2 Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite 5	SPA with hot module reload
Styling	Tailwind CSS 3	Utility-first responsive design
Animations	Framer Motion 11	Page transitions, card reveals, micro-interactions
Charts	Recharts 2	Radar chart for Startup Score
Icons	Lucide React	Consistent icon system
Backend	FastAPI 0.111	Async Python web framework
AI Orchestration	LangGraph StateGraph	Multi-agent pipeline with state management
Primary LLM	Gemini 2.5 Flash (Google GenAI)	All 6 agent calls + domain classification
Secondary LLM	Claude Sonnet (Anthropic)	HTML wireframe generation
Artifact Gen	python-pptx, zipfile	Pitch deck PPTX + code skeleton ZIP

2.3 Repository Structure

```
Daedalus-AI
├── backend
│   ├── main.py # FastAPI app + SSE + export endpoint
│   ├── requirements.txt # Python dependencies
│   ├── env.example # Environment variable template
│   ├── orchestrator/ # AI Agent Pipeline (Person A)
│   │   ├── __init__.py # Package exports: full pipeline, run_pipeline.py
│   │   ├── graph.py # LangGraph pipeline + dynamic fallback generator
│   │   ├── prompts.py # 5 agent system prompts (strict, open)
│   │   ├── agents.py # Gemini API call wrappers with retry logic
│   │   ├── critique.py # Round 2: Investor/Skeptic critique + revision routine
│   │   ├── scoring.py # Startup score computation (deterministic)
│   │   ├── domain_classifier.py # Idea → Domain classification entry point
│   │   ├── normalize.py # Output normalization for frontend contract
│   │   ├── fallback_filler.py # Idea-specific fallback data generator
│   │   ├── fallback_data.json # Static HRMS sample data
│   │   └── artifacts/ # Downloadable Export Generators (Person B)
│   │       ├── __init__.py
│   │       ├── pitch_deck.py # .pptx generator (python-pptx)
│   │       ├── wireframe_gen.py # .html wireframe (Claude API + template fallback)
│   │       └── code_skeleton.py # .zip FastAPI project scaffold
│   └── .
├── frontend
│   ├── index.html # Entry point
│   ├── package.json # Node dependencies
│   ├── vite.config.js # Vite configuration
│   ├── tailwind.config.js # Tailwind theme customization
│   ├── src
│   │   ├── main.jsx # React root mount
│   │   ├── App.jsx # Root component + layout orchestration
│   │   ├── App.css # Global styles
│   │   ├── index.css # Design system tokens
│   │   └── components
│   │       ├── AnimatedBackground.jsx # Particle/gradient background
│   │       ├── Navbar.jsx # Top navigation bar
│   │       ├── InputBar.jsx # Idea input + example chips
│   │       ├── AgentGrid.jsx # Agent card layout grid
│   │       ├── AgentCard.jsx # Individual agent result card
│   │       ├── CritiqueThread.jsx # Investor/Skeptic critique display
│   │       ├── ScoreRadar.jsx # Radar chart + score breakdown
│   │       ├── WireframePreview.jsx # Wireframe iframe preview
│   │       ├── ExportButtons.jsx # Download pitch deck, wireframe, code
│   │       ├── LoadingOverlay.jsx # Streaming progress overlay
│   │       ├── StreamProgressbar.jsx # Linear-style top progress bar
│   │       └── Footer.jsx # Page footer
│   ├── hooks
│   │   ├── useSSE.js # SSE stream consumer hook
│   │   └── utils
│   │       ├── constants.js # Agent definitions, example ideas, API BASE
│   │       ├── helpers.js # Utility functions (sleep, etc.)
│   │       ├── tailorFallback.js # Client-side fallback data generator
│   │       └── wireframeGen.js # Client-side wireframe HTML renderer
│   └── .
├── env # API keys (Gemini)
├── .gitignore
├── testpipeline.py # CLI test script for pipeline
└── README.md # Project documentation
```

3. Detailed Pipeline Workflow

The system follows a **3-round pipeline** that simulates a real co-founder team debate:

3.1 Pipeline Flow Diagram

```
sequenceDiagram
    participant U as User
    participant FE as React Frontend
    participant API as FastAPI Backend
    participant DC as Domain Classifier
    participant PM as PM Agent
    participant UI as UI Agent
    participant BE as Backend Agent
    participant MK as Marketing Agent
    participant INV as Investor Agent
    participant SK as Skeptic Agent
    participant SC as Scoring Engine
    participant AG as Artifact Generators

    U->>FE: Submit startup idea
    FE->>API: POST /build-startup {idea}
    API-->>FE: SSE stream begins

    Note over API,DC: Round 0: Domain Classification
    API->>DC: classify_domain(idea)
    DC-->>API: {domain, audience, problem, constraints, keywords}

    Note over API,MK: Round 1: Agent Analysis (PM first, then parallel)
    API->>PM: run_pm(idea + context)
    PM-->>API: {problem, solution, mvp_features, ...}
    API-->>FE: SSE: round1.pm data

    par Parallel Agent Calls
        API->>UI: run_ui(idea + context + pm)
        API->>BE: run_backend(idea + context + pm)
        API->>MK: run_marketing(idea + context + pm)
    end
    UI-->>API: {screens, design_system, flows}
    BE-->>API: {architecture, entities, api_endpoints}
    MK-->>API: {positioning, channels, pricing}
    API-->>FE: SSE: round1.ui, round1.backend, round1.marketing

    Note over API,SK: Round 2: Critique & Revision
    API->>INV: run_investor(all round1 data)
    API->>SK: run_skeptic(all round1 data)
    INV-->>API: {concerns, investment_thesis}
    SK-->>API: {flags, killer_questions}

    Note over API: Revision: Flagged agents re-run with critique
    API->>PM: revise_agent("pm", prior + critique)
    PM-->>API: Revised PM output
    API-->>FE: SSE: round2_critique data

    Note over API,SC: Round 3: Scoring
    API->>SC: compute_startup_score(final_state)
    SC-->>API: {feasibility, market_size, differentiation, team_fit}
    API-->>FE: SSE: startup_score

    Note over API,AG: Background: Artifact Generation
    par Artifact Generation
        API->>AG: generate_pitch_deck(result)
        API->>AG: generate_wireframe(ui_spec)
        API->>AG: generate_code_skeleton(backend_spec)
    end

    API-->>FE: SSE: session_id
    API-->>FE: SSE: [DONE]
```

3.2 Pipeline Stages in Detail

Stage 0: Domain Classification

- **Module:** `domain_classifier.py`
- **Purpose:** Classify the raw startup idea into a domain and extract structured context
- **Input:** Raw idea string
- **Output:** `{domain, audience, problem, constraints, keywords}`
- **LLM Call:** Gemini 2.5 Flash via `agents.classify_domain()`

Stage 1: Round 1 — Agent Analysis

- **Module:** `graph.py` → `_run_round1()`
- **Execution Order:** PM runs first (sequential), then UI + Backend + Marketing run **in parallel** via `asyncio.gather()`
- **Why PM First:** The PM output (problem, solution, MVP features) feeds into the other 3 agents as shared context

Agent	Prompt File	Output Schema
Product Manager	<code>PM_AGENT_PROMPT</code>	<code>{problem, target_users, solution, mvp_features, user_journey, success_metrics, risks}</code>
UI Designer	<code>UI_AGENT_PROMPT</code>	<code>{design_goal, screens[], information_architecture, visual_style, wireframe_notes}</code>
Backend Architect	<code>BACKEND_AGENT_PROMPT</code>	<code>{architecture, entities[], api_endpoints[], integrations, data_privacy, technical_risks}</code>
Marketing	<code>MARKETING_AGENT_PROMPT</code>	<code>{positioning, value_props, channels, launch_plan, competitors, messaging}</code>

Stage 2: Round 2 — Critique & Revision

- **Module:** `critique.py`
- **Process:** 1. **Investor Agent** reviews all Round 1 outputs and raises severity-tagged concerns 2. **Skeptic Agent** stress-tests for unrealistic assumptions and missing details 3. Concerns are **grouped by target agent** (PM, UI, Backend, Marketing) 4. Flagged agents are **re-invoked** with their original output + critique as input 5. Revised outputs replace the originals in the final state

Agent	Role	Output Schema
Investor	Seed investor evaluation	<code>{concerns[{agent, severity, issue, recommended_revision}], investment_thesis, ask_readiness}</code>
Skeptic	Devil's advocate	<code>{flags[{agent, severity, issue, recommended_revision}], killer_questions, overall_risk}</code>

Stage 3: Scoring

- **Module:** `scoring.py`

- **Algorithm:** Deterministic formula-based scoring (not LLM-generated)
- **Factors:**
 - `feasibility` = base 6 + MVP feature count bonus + endpoint count bonus – concern penalty
 - `market_size` = base 6 + channel count bonus + competitor count bonus – concern penalty
 - `differentiation` = base 5 + value prop bonus + success metrics bonus – concern penalty
 - `team_fit` = base 6 + risk identification bonus – concern penalty
- **Severity Penalties:** `{low: 0.5, medium: 1.0, high: 2.0}`
- **Range:** All scores clamped to `[1, 10]`

Stage 4: Artifact Generation (Background)

Runs asynchronously via `asyncio.create_task()` after the SSE stream completes:

1. **Pitch Deck** → 12-slide `.pptx` with dark navy theme
2. **Wireframe** → Interactive `.html` with Tailwind CSS
3. **Code Skeleton** → `.zip` with FastAPI scaffold (main.py, models.py, routes.py, database.py, docker-compose.yml)

4. Feature Documentation

4.1 Multi-Agent AI Pipeline

The core feature: 6 specialized AI agents powered by **Gemini 2.5 Flash** debate and refine a startup plan.

Key Technical Details: - Each agent receives a **strict JSON-only system prompt** (`prompts.py`) that enforces structured output - The `response_mime_type="application/json"` config forces Gemini to return valid JSON - **Retry Logic:** Exponential backoff (2 seconds) up to 5 retries for HTTP 429/503 errors - **Per-Agent API Keys:** Each agent can have its own Gemini API key (`GEMINI_PM_API_KEY`, etc.) or share a common `GEMINI_API_KEY` - **Fallback Safety:** If any agent fails, its output is replaced by a domain-tailored fallback

4.2 Real-Time SSE Streaming

The frontend receives results progressively via Server-Sent Events, enabling a "watching the team think" experience.

SSE Event Schema:

```
// Agent thinking placeholder
data: {"round": {"pm": {"streaming": true}}}

// Agent result
data: {"round": {"pm": {"problem": "...", "solution": "...", ...}}}

// Critique results
data: {"round": {"critique": {"investor_concerns": [...], "skeptical_flags": [...], ...}}}

// Startup score
data: {"startup_score": {"feasibility": 7, "market_size": 8, ...}}

// Session ID for exports
data: {"session_id": "uuid-string"}

// Stream end
data: [DONE]
```

Frontend SSE Consumer (`useSSE.js`): - Uses `fetch()` with `ReadableStream` reader (not `EventSource`) for POST request support - Implements chunk-by-chunk deep merge via `mergeChunk()` - Progress estimation from 0–100% based on data completeness - Automatic **abort + reset** capability - **Fallback mode:** If the backend is unreachable, the frontend simulates the stream locally with realistic delays using `tailorFallbackForIdea()`

4.3 Domain-Aware Fallback System

A sophisticated offline fallback system that generates realistic, domain-specific responses without any API calls.

Supported Domains (keyword-matched):

Domain	Trigger Keywords	Primary Color
Healthcare	health, doctor, hospital, patient, medical	#06B6D4
E-commerce & Retail	shop, fashion, clothes, retail, marketplace	#EC4899
EdTech	learn, school, skill, education, course	#8B5CF6
FinTech	money, invest, finance, stock, crypto	#3B82F6
Logistics & Mobility	car, vehicle, ev, fleet, logistics	#14B8A6
FoodTech	food, kitchen, restaurant, recipe	#F97316
ClimateTech	carbon, green, climate, energy, solar	#22C55E
SaaS / Digital Services	(default fallback)	#7C3AED

Each domain generates: - Custom screens, user flows, and color schemes - Domain-specific database schemas and API endpoints - Realistic investor concerns and skeptic flags - Market sizing (TAM/SAM/SOM) data - Tailored pricing tiers and GTM channels

4.4 Pitch Deck Generator

Module: `pitch_deck.py`

Generates a **12-slide widescreen (16:9)** `.pptx` presentation with a dark navy professional theme.

Slide #	Title	Content Source
1	Cover	Idea name + tagline + domain badge
2	The Problem	<code>pm.problem</code>
3	Our Solution	<code>pm.solution</code>
4	MVP Features	<code>pm.mvp_features</code> list
5	Who We Serve	<code>pm.target_users</code> + <code>marketing.icp</code> (two-column)
6	Product Screens	<code>ui.screens</code> names
7	Technology	<code>backend.tech_choices</code> + <code>backend.architecture</code>

8	Market Opportunity	<code>marketing.tam_sam_som</code> + <code>marketing.positioning</code>
9	Go-To-Market	<code>marketing.channels</code> + <code>marketing.pricing</code> (two-column)
10	Risks & Mitigations	<code>round2_critique</code> concerns + PM revisions (two-column)
11	Startup Score	Score metrics as color-coded cards (green ≥7, indigo ≥5, red <5)
12	CTA	"Ready to build the right thing?" closing slide

Design Features: - Brand colors: Indigo (#6366F1) primary, Emerald (#10B981) accent - Dark navy background (#0F172A) with accent left-bar on each slide - Custom text styling with `RGBColor` and `Pt` font sizing

4.5 Interactive Wireframe Generator

Module: `wireframe_gen.py`

Two-tier generation strategy:

Claude-Powered (if `ANTHROPIC_API_KEY` is set): - Calls Claude Sonnet to generate a pixel-perfect HTML page - Prompt instructs: sidebar nav, main content, at least one modal, realistic data - Uses the design system colors from UI spec

Template Fallback (no API required): - Renders a rich, interactive SaaS wireframe using structured data - Components rendered based on keyword matching:

- `"kpi"` / `"card"` → KPI cards grid (4 cards with realistic metrics)
- `"table"` / `"list"` → Data table with 5 rows, status badges, priority tags
- `"calendar"` → Month calendar with color-coded events
- `"chart"` / `"analytics"` → Bar chart with 12 monthly data points
- `"search"` / `"filter"` → Search bar with filters dropdown
- `"modal"` / `"panel"` → Detail panel with actions
- Sidebar navigation with screen switching via JavaScript
- Dark mode (slate-900) with the design system's primary/accent colors
- Google Fonts integration matching the UI spec

4.6 Code Skeleton Generator

Module: `code_skeleton.py`

Generates a downloadable `.zip` containing a complete **FastAPI project scaffold**:

File	Description
<code>main.py</code>	FastAPI app with CORS, router include, health check
<code>models.py</code>	SQLAlchemy ORM models + Pydantic schemas from database schema

<code>routes.py</code>	API router stubs for every endpoint (with <code>raise HTTPException(501)</code>)
<code>database.py</code>	SQLAlchemy engine + session setup + optional Redis client
<code>requirements.txt</code>	Dependencies (conditionally includes JWT, Redis, scikit-learn)
<code>.env.example</code>	Environment template with all required variables
<code>docker-compose.yml</code>	Docker setup with PostgreSQL + optional Redis
<code>README.md</code>	Setup guide with architecture, endpoints, getting started

Smart Field Type Mapping: - Fields ending in `_at` or containing `date` → `DateTime` - Fields ending in `_id` → `Integer` (foreign key) - Fields named `status`, `type`, `role` → `String(50)` - Fields named `salary`, `total_amount` → `Float` - All others → `String(255)`

4.7 Output Normalization Layer

Module: `normalize.py`

A critical middleware layer that transforms raw Gemini output into a **stable frontend display contract**. Handles the impedance mismatch between live LLM output (variable key names, nested objects) and the frontend's expected data shapes.

Key normalizations: - **Colors:** Named colors (e.g., "indigo") → hex values (#6366F1) via 18-entry color map - **Fonts:** Extracts font family names from prose (e.g., "Clean sans-serif using Inter" → "Inter") - **Screens:** Handles both `{name, components}` dicts and plain strings - **Endpoints:** Normalizes `{method, path, purpose}` objects to `"METHOD /path - purpose"` strings - **Critique items:** Flattens `{agent, severity, issue, recommended_revision}` dicts to human-readable strings - **Marketing messaging:** Combines `tagline` and `elevator_pitch` into single string

4.8 Frontend UI Components

Component	File	Purpose
InputBar	<code>InputBar.jsx</code>	Text input with example idea chips, loading state, stop button
AgentGrid	<code>AgentGrid.jsx</code>	Responsive grid layout for 4 Round 1 agent cards
AgentCard	<code>AgentCard.jsx</code>	Expandable card with shimmer loading, markdown rendering
CritiqueThread	<code>CritiqueThread.jsx</code>	Investor concerns + Skeptic flags + revision display
ScoreRadar	<code>ScoreRadar.jsx</code>	Recharts radar chart + animated score counters
WireframePreview	<code>WireframePreview.jsx</code>	Iframe-embedded wireframe preview (server or client-rendered)

ExportButtons	ExportButtons.jsx	Download buttons for pitch deck, wireframe, code skeleton
StreamProgressBar	StreamProgressBar.jsx	Linear-style top progress bar during streaming
LoadingOverlay	LoadingOverlay.jsx	Full-screen overlay with agent completion indicators
AnimatedBackground	AnimatedBackground.jsx	Particle/gradient ambient background

5. API Documentation

5.1 API Endpoints Overview

Method	Endpoint	Description	Auth
GET	/	API information & link to docs	None
GET	/health	Server status + orchestrator availability check	None
POST	/build-startup	Start the multi-agent pipeline, returns SSE stream	None
GET	/export/pitch-deck?id=	Download .pptx pitch deck	None
GET	/export/wireframe?id=	Download .html wireframe	None
GET	/export/code-skeleton?id=	Download .zip code skeleton	None

5.2 Endpoint Details

GET /

Purpose: Root endpoint — returns API metadata.

Response 200 OK:

```
{
  "name": "Daedalus AI Co-Founder API",
  "version": "1.0.0",
  "status": "running",
  "docs": "http://localhost:8000/docs",
  "health": "http://localhost:8000/health"
}
```

GET /health

Purpose: Health check — confirms the server is running and whether the AI pipeline is available.

Response 200 OK:

```
{
  "status": "ok",
  "orchestrator_available": true,
  "mode": "live"
}
```

Field	Type	Description
status	string	Always "ok" if the server is responding
orchestrator_available	boolean	true if the LangGraph pipeline + Gemini agents are importable
mode	string	"live" (real AI pipeline) or "mock" (fallback data)

POST /build-startup

Purpose: The primary endpoint — accepts a startup idea and returns a Server-Sent Events stream with progressive results from all 6 AI agents.

Request Body:

```
{
  "idea": "AI-powered telemedicine app for rural India"
}
```

Field	Type	Required	Validation
idea	string	Yes	Must be non-empty after trimming whitespace

Response: 200 OK — text/event-stream

Headers:

```
Content-Type: text/event-stream
Cache-Control: no-cache
X-Accel-Buffering: no
Connection: keep-alive
```

SSE Event Sequence:

```
data: {"round":1,"pm":{"streaming":true}}
data: {"round":1,"ui":{"streaming":true}}
data: {"round":1,"backend":{"streaming":true}}
```

```
[
  "data": { "round": { "marketing": { "streaming": true } } },
  "data": { "round": { "pm": { "problem": "x", "solution": "y", "target_users": [ "a" ], "mvp_features": [ "a" ], "user_journey": [ "a" ], "success_metrics": [ "a" ] } } },
  "data": { "round": { "ui": { "design_goal": "x", "screens": [ "a" ], "visual_style": [ "a" ], "wireframe_notes": [ "a" ] } } },
  "data": { "round": { "backend": { "architecture": "x", "entities": [ "a" ], "api_endpoints": [ "a" ], "integrations": [ "a" ] } } },
  "data": { "round": { "marketing": { "positioning": "x", "value_props": [ "a" ], "channels": [ "a" ], "messaging": [ "a" ] } } },
  "data": { "rounds_critique": { "investor_concerns": [ "a" ], "skeptics_flags": [ "a" ], "investment_thesis": "x", "killer_questions": [ "a" ], "revisions": [ "a" ] } },
  "data": { "startup_score": { "feasibility": 7, "market_size": 8, "differentiation": 6, "team_fit": 7 } },
  "data": { "session_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890" },
  "data": { "done": true }
]
```

Error Response 400 Bad Request:

```
[
  "detail": "Idea cannot be empty"
]
```

Behavior Details: - If `orchestrator_available == true`: Runs the live Gemini-powered pipeline (`_live_stream`) - If `orchestrator_available == false`: Runs the domain-tailored mock pipeline (`_mock_stream`) with realistic delays - If the live pipeline crashes mid-execution: Automatically falls back to mock stream - Artifact generation (pitch deck, wireframe, code skeleton) begins as a background task after the stream completes

GET /export/pitch-deck?id={session_id}

Purpose: Download the generated PowerPoint pitch deck.

Query Parameters:

Parameter	Type	Required	Description
id	string (UUID)	Yes	Session ID returned by /build-startup

Response 200 OK:

```
Content-Type: application/vnd.openxmlformats-officedocument.presentationml.presentation
Content-Disposition: attachment; filename="daedalus_pitch_deck.pptx"
```

Error Response 404 Not Found:

```
[
  "detail": "Session not found"
]
```

Behavior: 1. Checks if `pitch_deck.pptx` already exists in `/tmp/ai-cofounder-sessions/{id}/` 2. If not, regenerates on-the-fly from the stored session data 3. Returns the file as a downloadable attachment

GET /export/wireframe?id={session_id}

Purpose: Download the generated HTML wireframe.

Query Parameters:

Parameter	Type	Required	Description
<code>id</code>	string (UUID)	Yes	Session ID returned by /build-startup

Response 200 OK:

```
Content-Type: text/html
Content-Disposition: attachment; filename="daedalus_wireframe.html"
```

Error Response 404 Not Found:

```
{
  "detail": "Session not found"
}
```

Behavior: 1. Checks if `wireframe.html` exists in the session directory 2. If not, regenerates using Claude API (preferred) or template fallback 3. Returns the HTML file as a downloadable attachment

`GET /export/code-skeleton?id={session_id}`

Purpose: Download the generated FastAPI code skeleton as a ZIP archive.

Query Parameters:

Parameter	Type	Required	Description
<code>id</code>	string (UUID)	Yes	Session ID returned by /build-startup

Response 200 OK:

```
Content-Type: application/zip
Content-Disposition: attachment; filename="daedalus_code_skeleton.zip"
```

Error Response 404 Not Found:

```
{
  "detail": "Session not found"
}
```

Behavior: 1. Checks if `code_skeleton.zip` exists in the session directory 2. If not, regenerates from the stored backend spec 3. ZIP contains: `main.py`, `models.py`, `routes.py`, `database.py`, `requirements.txt`, `.env.example`, `docker-compose.yml`, `README.md`

6. Internal API — Gemini Agent Calls

All agent calls flow through `agents.py` → `call_agent()`:

```
def call_agent(
    system_prompt: str,          # From prompts.py
    input_json: dict,           # Context + prior output
    agent_name: str,            # For API key lookup
    model: str = "gemini-2.5-flash"
):
    # ... (implementation details) ...
```

Call Configuration: | Parameter | Value | |-----|-----| | Model | `gemini-2.5-flash` (configurable via `GEMINI_MODEL` env) | | Temperature | `0.4` | | Response MIME type | `application/json` | | Max retries | `5` | | Retry codes | `429` (rate limit), `503` (service unavailable) | | Backoff | Exponential: 2, 4, 8, 16, 32 seconds |

JSON Extraction: Handles markdown-fenced responses (````json ... ````) and extracts JSON via regex as a fallback.

7. Data Flow & Output Contract

7.1 Complete Pipeline Output Schema

```
{
  "idea": "string - original user idea",
  "domain": "string - classified domain",
  "idea_context": {
    "domain": "string",
    "audience": "string",
    "problem": "string",
    "constraints": ["string"],
    "keywords": ["string"]
  },
  "round": 1,
  "pm": {
    "problem": "string",
    "solution": "string",
    "target_users": ["string"],
    "mvp_features": ["string"],
    "core_features": ["string"],
    "user_journey": ["string"],
    "roadmap": ["string"],
    "success_metrics": ["string"],
    "risks": ["string"],
    "tagline": "string",
    "mvp_scope": "string"
  },
  "ui": {
    "design_system": {
      "primary_color": "#hex",
      "accent_color": "#hex",
      "font": "string",
      "style": "string"
    },
    "screens": [
      { "name": "string", "components": ["string"] },
      ...
    ],
    "key_user_flows": ["string"],
    "key_interactions": ["string"],
    "wireframe_notes": ["string"],
    "component_library": "string"
  },
  "backend": {
    "architecture": "string",
    "services": ["string"],
    "endpoints": [{"name": "string", "fields": ["string"]}],
    "api_endpoints": ["string"],
    "tech_choices": {"auth": "string", "queue": "string", "infra": "string"},
    "tech_stack": ["string"],
    "integrations": ["string"]
  }
}
```

```

    "data_privacy": ["string"],
    "technical_risks": ["string"],
  },
  "marketing": {
    "positioning": "string",
    "competitive_positioning": "string",
    "value_props": ["string"],
    "channels": ["string"],
    "launch_plan": ["string"],
    "competitors": ["string"],
    "messaging": "string",
    "gta_strategy": "string",
    "target_cac": "string",
    "target_ltv": "string"
  },
  },
  "round2_critique": {
    "investor_concerns": ["string - flattened from structured objects"],
    "skeptical_flags": ["string - flattened from structured objects"],
    "investment_thesis": "string",
    "ask_readiness": "low|medium|high",
    "killer_questions": ["string"],
    "overall_risk": "low|medium|high",
    "revisions": {
      "pm": {
        "revised_mvp_scope": "string",
        "differentiation": "string",
        "updated_roadmap": ["string"]
      }
    }
  },
  "startup_score": {
    "feasibility": "int 1-10",
    "market_size": "int 1-10",
    "differentiation": "int 1-10",
    "team_fit": "int 1-10"
  }
}
```

8. Environment Configuration

8.1 Required Environment Variables

Variable	Purpose	Required
GEMINI_API_KEY	Shared Gemini API key for all agents	Yes (if agent-specific keys not set)
GEMINI_PM_API_KEY	PM Agent Gemini key	Optional
GEMINI_UI_API_KEY	UI Agent Gemini key	Optional
GEMINI_BACKEND_API_KEY	Backend Agent Gemini key	Optional
GEMINI_MARKETING_API_KEY	Marketing Agent Gemini key	Optional
GEMINI_INVESTOR_API_KEY	Investor Agent Gemini key	Optional
GEMINI_SKEPTIC_API_KEY	Skeptic Agent Gemini key	Optional

GEMINI_DOMAIN_CLASSIFIER_API_KEY	Domain Classifier Gemini key	Optional
ANTHROPIC_API_KEY	Claude API key for wireframe generation	Optional
GEMINI_MODEL	Override default model (default: <code>gemini-2.5-flash</code>)	Optional
CORS_ORIGINS	Comma-separated allowed origins	Optional
VITE_API_URL	Frontend API base URL (default: <code>http://localhost:8000</code>)	Optional

8.2 CORS Configuration

Default allowed origins: - <http://localhost:5173> (Vite dev server) - <http://localhost:3000> - <http://localhost:5174>

9. Running the Project

9.1 Backend

```
cd backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # Add GEMINI_API_KEY
uvicorn main:app --reload --port 8000
```

API Docs: <http://localhost:8000/docs> (Swagger UI auto-generated by FastAPI)

9.2 Frontend

```
cd Frontend
npm install
npm run dev # → http://localhost:5173
```

9.3 Test Pipeline (CLI)

```
# From project root
python testpipeline.py
```

This runs the full pipeline programmatically with a healthcare startup idea and prints the JSON result.

10. Resilience & Error Handling

Failure Scenario	Handling Strategy
------------------	-------------------

Gemini API rate limit (429)	Exponential backoff retry up to 5 attempts
Gemini API down (503)	Same retry strategy
Single agent fails	Individual agent fallback from <code>taylor_fallback_for_idea()</code>
Entire pipeline fails	Full fallback to domain-tailored mock data
Backend unreachable	Frontend runs client-side fallback with simulated delays
Artifact generation fails	Logged but doesn't crash the SSE stream
Invalid Gemini JSON response	Regex extraction of <code>{...}</code> from response text
Empty Gemini response	<code>AgentCallError</code> raised, caught by fallback handler
Anthropic API fails (wireframe)	Falls back to template-rendered HTML wireframe

11. Team Roles & Module Ownership

Person	Role	Owns
Person A	Agent Logic & Orchestration	<code>backend/orchestrator/</code> — <code>graph.py</code> , <code>prompts.py</code> , <code>agents.py</code> , <code>critique.py</code> , <code>scoring.py</code>
Person B	Backend API & Artifact Generation	<code>backend/main.py</code> , <code>backend/artifacts/</code> — <code>pitch_deck.py</code> , <code>wireframe_gen.py</code> , <code>code_skeleton.py</code>
Person C	Frontend & Demo Experience	<code>Frontend/src/</code> — all components, hooks, <code>utils</code>

12. Key Design Decisions

- PM-First Sequential Execution:** The PM agent runs before UI, Backend, and Marketing because its output (problem statement, solution, MVP features) serves as essential context for the other agents. The remaining 3 run in parallel for speed.
- Deterministic Scoring:** Startup scores are computed algorithmically from pipeline output counts and severity penalties — not by an LLM. This ensures reproducibility and prevents hallucinated scores.
- Dual-Mode Wireframe Generation:** The wireframe generator tries Claude for high-fidelity output but falls back to a sophisticated template engine, ensuring the feature works even without an Anthropic API key.
- SSE over WebSockets:** SSE was chosen over WebSockets because the data flow is unidirectional (server → client), simpler to implement, and automatically handles reconnection.

Frontend Fallback Independence: The frontend can operate entirely offline using `taiorFallback.js`, making demos possible without any running backend. The client-side fallback matches the same domain detection logic as the server.

Session-Based Artifact Storage: Artifacts are stored under `/tmp/ai-cofounder-sessions/{session_id}/` and lazily generated on first export request if the background pre-generation hasn't completed yet.

Report Generated: July 16, 2026
Project: Daedalus-AI — AI Co-Founder Team
Total Files Analyzed: 30+
Total Lines of Code: ~5,500+ (backend) + ~3,000+ (frontend)